Game of 24

S.P. Wong & J. Huiszoon
s.p.wong@student.rug.nl & j.huiszoon@student.rug.nl

October 27, 2026

1 Problem description

This problem requires a programme that solves 'the Game of 24'. The user provides four integer numbers between 1 and 10 (based on a full-deck of cards, without the jack, queen, king and joker). The programme must determine whether these numbers are able to be combined to obtain the total value of 24, using only four arithmetic operations: addition, subtraction, multiplication and division.

Each of these four numbers must be used exactly once, while parentheses can be used to determine the order of the operations. If a valid solution exists, the programme should print the mathematical expression that equals 24. If no solution exists, the programme should inform the user that there is no solution possible.

2 Problem analysis

We can interpret the problem as follows: A set of given numbers can be turned into a valid equation that equals 24 if it can be split into 2 sub-equations that when joined with an operator will equal 24. That is a set A can be turned into a valid equation a if $B, C \subset A$, $|C| < |B|$, and either

- $a = b + c$
- $a = b - c$
- $a = b \cdot c$
- or $a = \frac{b}{c}$ where $b \bmod c = 0$

where b and c are equations consisting of only the elements in B and C and the basic operators.

Equations can continue to be split up like this until either $|A| = 1$, where a is equal to the only element of A, or until we cannot find a valid equation for b and c such that it equals a. By manipulating the equation, assuming we know a and c, we can find b where b is equal to one of the following;

- $a - c = b$
- $a + c = b$
- $\frac{a}{c} = b$ where c is not 0
- $a \cdot c = b$ where $b \bmod c = 0$

By setting a equal to 24, and iterating c through all possible values, we can find what b must equal for c to be valid. We can then repeat the process, this time setting a to what we found for b, and repeat until we find valid values for b and c.

3 Design

Due to the recursive nature of the program, as well as to keep the complexity of printing the final equation simple, we elected to use a node tree to represent the equations in this program. Each node is either an equation or a number. Equation nodes contain a pointer to a left and right node, as well as an operator. Number nodes only contain a number.

First the 4 numbers in set A are read from the standard input and stored in the array `numbers`. The function calculateEquation is called with the `size of numbers`, `numbers`, and the `goal` of `24`. We check if the result is a NULL pointer, and print "No equation" if so. Otherwise, we print the resulting equation and clean up the allocated memory before we terminate.

In the function calculateEquation, we first check if the size of `numbers` is 1. If the `size` is 1 and the `goal` equals the `number`, then we construct a new number node set to the value of the `number`. If it does not, then we return NULL, indicating that there is no such equation that can equal the goal.

If the size of numbers is not 1, then we set c (stored in variable `right`) to the first number in the array. We also create a variable called `operator` to store the final operator we need to construct a. We check to see if there is a b (stored in variable `left`) which satisfies one of the equations we listed in the analysis. We do this by plugging in a and c into each of our equations, then recursively calling calculateEquation with b as our new `goal`, and `numbers` excluding the numbers we used for c. If we found a valid b, then we set `right` to the returned node pointer and update `operator` to the corresponding operation. If we fail to find an equation b, then we rotate the array and try again with the next number for c.

Should we fail to find a value for b when c is only a number node, we construct a new equation of the first 2 elements of `numbers` and set c to it. We then repeat the process of rotating through the numbers, this time also rotating through the operators as well. We will continue this process until $|C| > |B|$, in which case there is no possible solution.

After concluding all possible iterations, then if there is no valid b that exists, then a does not exist either. We clean up the memory that we used, then return NULL to indicate a failed search. If b does exist, then we return a new equation constructed using the variables `left`, `right`, and `operator`.

4 Program code

gameof24.c

```
1  #include <stdlib.h>
2  #include <stdio.h>
3
4  struct node;
5
6  typedef struct expression {
7      char operator;
8      struct node* left;
9      struct node* right;
10 } expression;
11
12 typedef struct node {
13     int isExpression;
14     union wrapper {
15         expression expr;
16         int number;
17     } value;
18 } node;
19
20 void printEquation(node* root) {
```

```
21
22      if (root->isExpression) {
23          printf("(");
24
25          printEquation(root->value.expr.left);
26
27          printf("%c", root->value.expr.operator);
28
29          printEquation(root->value.expr.right);
30
31          printf(")");
32      } else {
33          printf("%d", root->value.number);
34      }
35
36  }
37
38  node* constructNumber(int number) {
39      node* n = malloc(sizeof(node));
40      n->isExpression = 0;
41      n->value.number = number;
42      return n;
43  }
44
45  char getNextOperator(char operator) {
46      switch(operator) {
47          case '+': return '-';
48          case '-': return '*';
49          case '*': return '/';
50          case '/': return '+';
51      }
52  }
53
54  // Get the value of an equation
55  int getValue(node* root) {
56      if (!root->isExpression) {
57          return root->value.number;
58      }
59
60      // Get the value of the left side
61      int a = getValue(root->value.expr.left);
62      // Get the value of the right side
63      int b = getValue(root->value.expr.right);
64
65      // Calculate the value
66      switch (root->value.expr.operator) {
67          case '+': return a + b;
68          case '-': return a - b;
69          case '*': return a * b;
70          case '/': return a / b;
71      }
72  }
73
74  int setNumbers(node* root, int size, int numbers[]) {
75      if (size == 0) {
76          return 0;
77      }
78
```

```
79      if (root->isExpression) {
80          // Set leftSize = the amount of numbers on the left side
81          int leftSize = setNumbers(root->value.expr.left, size, numbers);
82          // Set rightSize = the amount of numbers on the right size
83          int rightSize = setNumbers(root->value.expr.right, size - leftSize, &numbers[
                leftSize]);
84          return leftSize + rightSize;
85      } else {
86          root->value.number = numbers[0];
87          return 1;
88      }
89  }
90
91  int isValidEquation(node* root) {
92      if (!root->isExpression) {
93          return 1;
94      }
95
96      int left = isValidEquation(root->value.expr.left);
97      int right = isValidEquation(root->value.expr.left);
98      int isValid = left && right;
99
100     if (root->value.expr.operator == '/') {
101         int lv = getValue(root->value.expr.left);
102         int rv = getValue(root->value.expr.right);
103         isValid = isValid && lv % rv == 0;
104     }
105
106     return isValid;
107 }
108
109 void freeEquation(node* root) {
110     // If the node is an expression, first free the left and right
111     if (root->isExpression) {
112         freeEquation(root->value.expr.left);
113         freeEquation(root->value.expr.right);
114     }
115
116     free(root);
117 }
118
119 // Shift all elements to the left by 1, and move the first to the end
120 void rotate(int size, int numbers[]) {
121     int first = numbers[0];
122     for (int i = 1; i < size; i++) {
123         numbers[i - 1] = numbers[i];
124     }
125     numbers[size - 1] = first;
126 }
127
128 node* calculateEquation(int size, int numbers[], int goal) {
129     // If there is only 1 element left, return it if it is equal to goal
130     if (size == 1) {
131         if (goal != numbers[0]) {
132             return NULL;
133         }
134         return constructNumber(numbers[0]);
135     }
```

```
136
137     // Set the right equal to the first element
138     node *right = constructNumber(numbers[0]);
139     node *left = NULL;
140     char operator = '+';
141
142     // let k = the amount of numbers on the right side of the equation
143     for (int k = 1; k <= (size + 1) / 2; k++) {
144         for (int i = 0; i < size; i++) {
145             // Set the numbers on the right equation to the numbers in numbers[]
146             setNumbers(right, k, numbers);
147             if (!isValidEquation(right)) {
148                 rotate(size, numbers);
149                 continue;
150             }
151
152             int rValue = getValue(right);
153
154             // Addition
155             int subgoal = goal - rValue;
156             left = calculateEquation(size - k, &numbers[k], subgoal);
157             if (left != NULL) {
158                 operator = '+';
159                 break;
160             }
161
162             // Subtraction
163             subgoal = goal + rValue;
164             left = calculateEquation(size - k, &numbers[k], subgoal);
165             if (left != NULL) {
166                 operator = '-';
167                 break;
168             }
169
170             // Multiplication
171             if (rValue > 0 && goal % rValue == 0) {
172                 subgoal = goal / rValue;
173                 left = calculateEquation(size - k, &numbers[k], subgoal);
174                 if (left != NULL) {
175                     operator = '*';
176                     break;
177                 }
178             }
179
180             // Division
181             subgoal = goal * rValue;
182             left = calculateEquation(size - k, &numbers[k], subgoal);
183             if (left != NULL) {
184                 operator = '/';
185                 break;
186             }
187             rotate(size, numbers);
188         }
189
190         if (left != NULL) {
191             // Found valid equation
192             break;
193         } else {
```

```
            if (right->isExpression && right->value.expr.operator != '/') {
                right->value.expr.operator = getNextOperator(right->value.expr.operator);
                k--;
                continue;
            }

            // Add the another number to the right equation
            node* n = malloc(sizeof(node));
            n->isExpression = 1;
            n->value.expr.right = right;
            n->value.expr.left = constructNumber(numbers[1]);
            n->value.expr.operator = '+';
            right = n;
        }
    }

    // If we failed to find anything, clean up and return NULL
    if (left == NULL) {
        freeEquation(right);
        return NULL;
    }

    // Construct equation and return it
    node* n = malloc(sizeof(node));
    n->isExpression = 1;
    n->value.expr.left = left;
    n->value.expr.right = right;
    n->value.expr.operator = operator;

    return n;
}

int main(int argc, char* argv[]) {
    int numbers[4];
    for (int i = 0; i < 4; i++) {
        scanf("%d", &numbers[i]);
    }

    node* eq = calculateEquation(4, numbers, 24);
    if (eq == NULL) {
        printf("No equation\n");
    } else {
        printEquation(eq);
        freeEquation(eq);
    }
}
```

5 Test results

- Input: (normal case)

```
1 2 3 4
```

Output:

```
(((4*3)*2)*1)
```

- Input: (normal case)

```
3 6 7 5
```

Output:

```
(((7-5)+6)*3)
```

- Input: (normal case)

```
5 4 2 8
```

Output:

```
(((5*8)/2)+4)
```

- Input (sub-equation on the right side)

```
5 6 7 7
```

Output:

```
((5-(7/7))*6)
```

- Input: (no valid solution)

```
6 7 7 8
```

Output:

```
No equation
```

6 Evaluation

Due to the recursive design of our algorithm, the resulting program is quite readable and understandable. Given the analysis of the problem, recursion seemed like the most optimal choice. Provided with the constraints for the input, the program successfully produces a valid equation that equals 24, or successfully determines that no such equation exists. This program theoretically can also be used for targets other than 24, or for sets that contain more than 4 elements, although that would require more evaluation to prove true. Due to the nature of the algorithm, this program priorities sub-equations to be on the left size, and will only find a solution where an equation is on the right when it cannot find a satisfying equation on the left.

We choose a recursive approach to solve this problem. Another algorithm would be one that loops through all possible combinations of numbers and operators, although the time complexity of such a program is much larger than a recursive one. Such an algorithm would also need to take into account different positioning of parentheses, which would add more complexity as well.